AI native 线上 Bug 修复复盘 agent 技术方案

- 项目背景
  - 1. 经验无法复用
  - 2. 高价值信息无法沉淀
  - 3. 复盘成本高
- 项目目标
  - 1. 自动化结构化复盘
  - 2. 建立团队级踩坑知识库
  - 3. 降低重复排查成本
- 技术调研与选型
  - 1. 为什么不用传统规则引擎
    - 方案一：纯正则 + 关键词提取
    - 方案二：Embedding + 向量检索
- 整体架构设计
  - 系统分层
- 核心实现细节
- 1. 事件采集层
  - 数据来源
    - Git 平台
    - 日志系统
    - 飞书群消息
  - 为什么采用事件驱动
- 2. LLM 结构化抽取层
  - Prompt 工程设计
  - 标准输出结构
  - 为什么不用自由生成
  - 长上下文处理
- 3. RAG 知识增强层
  - 为什么需要 RAG

向量化内容
检索策略
纯向量检索问题
4. 审核后台
为什么必须人工审核
审核后台功能
技术实现
Diff 高亮
5. 自动发布链路
稳定性设计
1. 幂等设计
2. LLM 降级机制
3. 消息重试
4. 质量评分机制
遇到的核心问题
1. LLM 幻觉严重
解决方案
2. PR 信息噪声过大
解决方案
3. Token 成本问题
优化方案
日志裁剪
分层摘要
如何进一步优化
1. Agent 化演进
2. 引入 Code Graph
3. 多模型路由
项目亮点总结
1. 跑通 AI 工程闭环
2. 强调可控性
3. 有明确稳定性设计
面试里最容易被深挖的问题

1. 为什么 RAG 不直接用向量检索?
2. 如何降低 LLM 幻觉?
3. 为什么必须人工审核?
4. 如何做长上下文压缩?
5. 如何控制 Token 成本?
6. 如何保证知识库质量?
7. 为什么采用事件驱动架构?
8. 如果每天事故量暴涨如何扩容?

项目背景

业务 前端/后端 团队在处理线上 Bug 时，通常会经历：

- 线上告警触发
- 排查日志与代码变更
- 修复并发布
- 在 PR、飞书群、临时文档中同步问题结论

但真正有价值的信息，例如：

- Bug 根因
- 影响范围
- 修复策略
- 规避方案
- 类似问题排查经验

往往以非结构化形式散落在多个系统中。

导致几个典型问题：

1. 经验无法复用

同类问题重复出现时，仍需要重新排查。

例如：

- React 闭包导致状态失效
- SSE 长连接异常断流
- Promise 并发导致接口竞态

- WebSocket 重连风暴
- CI 环境变量污染

团队成员可能已经踩过坑，但缺乏统一检索入口。

2. 高价值信息无法沉淀

PR 评论、飞书聊天记录、发布备注具有很强时效性：

- 新人无法快速获取历史经验
- 排查经验依赖“人记忆”
- 团队知识无法形成长期资产

3. 复盘成本高

很多团队要求写事故复盘，但：

- 工程师不愿手工整理
- 文档格式不统一
- 信息缺失严重
- 最终 Wiki 可读性差

因此需要一条自动化知识沉淀管道。

项目目标

建设 AI 原生的 Bug 复盘沉淀系统，实现：

```
PR / 日志 / 群聊
        ↓
事件检测
        ↓
LLM结构化抽取
        ↓
知识增强RAG
        ↓
人工审核
        ↓
自动发布Wiki
        ↓
团队踩坑知识库
```

核心目标：

1. 自动化结构化复盘

将零散信息自动生成标准化内容：

- 问题现象
- 根因分析
- 修复方案
- 风险影响
- 规避建议
- 关联模块

2. 建立团队级踩坑知识库

支持：

- 语义搜索
- 相似问题召回
- 历史案例推荐
- Agent 自动辅助排查

3. 降低重复排查成本

目标指标：

- Bug 沉淀时效 < 1 天
- 审核耗时 < 3 分钟
- 同类问题定位时间下降 30%+

技术调研与选型

1. 为什么不用传统规则引擎

初期调研过：

方案一：纯正则 + 关键词提取

例如：

```
fix:
bug:
hotfix:
线上问题:
```

问题：

- 语义表达不统一
- 大量漏检
- 无法抽取上下文
- 无法总结根因

最终只能做“信息收集”。

方案二：Embedding + 向量检索

只能做：

- 相似案例召回

但无法：

- 自动总结

- 生成结构化复盘
- 归纳根因

因此最终采用：

```
LLM + 轻量RAG + 人工审核
```

混合架构。

整体架构设计

系统分层

```
事件采集层
    ↓
任务编排层
    ↓
LLM抽取层
    ↓
RAG知识增强层
    ↓
审核发布层
    ↓
知识库检索层
```

核心实现细节

1. 事件采集层

数据来源

统一监听：

Git 平台

监听：

- PR merged
- commit message
- release tag

通过 Webhook 接入。

日志系统

接入：

- Sentry
- ELK
- Loki

自动关联：

- error stack
- release version
- affected module

飞书群消息

监听：

- “已修复”
- “hotfix”
- “线上事故”
- “回滚”

等高频语义。

为什么采用事件驱动

因为：

- 解耦
- 易扩展

- 支持异步重试

整体基于：

```
Webhook + 消息队列
```

实现。

队列层可选：

- Kafka
- RabbitMQ
- Redis Stream

中小团队最终采用 Redis Stream。

原因：

- 部署成本低
- 延迟低
- 支持消费组
- 足够支撑当前量级

2. LLM 结构化抽取层

Prompt 工程设计

核心问题：

LLM 容易：

- 幻觉
- 输出格式漂移
- 漏字段

因此采用：

```
Schema约束 + Few-shot + JSON输出
```

标准输出结构

统一抽取：

```JSON
{
  "title": "",
  "root_cause": "",
  "impact": "",
  "fix_solution": "",
  "prevention": "",
  "related_modules": [],
  "keywords": []
}
```

为什么不用自由生成

因为：

- Wiki 结构无法统一
- 后续无法做索引
- 无法沉淀向量知识

结构化输出更适合：

- RAG
- 搜索
- BI统计
- Agent推理

长上下文处理

实际问题：

一次事故可能包含：

- PR
- 多条日志
- 多轮聊天记录

Token 很容易爆。

因此采用：Map-Reduce Summarization

流程：

```
日志分块
    ↓
局部摘要
    ↓
聚合总结
    ↓
结构化输出
```

避免：

- 上下文过长
- 推理成本过高
- 响应变慢

3. RAG 知识增强层

为什么需要 RAG

仅靠当前 Bug 信息：

LLM 无法知道：

- 历史类似事故
- 团队规范
- 历史最佳实践

因此引入轻量 RAG。

向量化内容

主要包括：

- 历史复盘
- 前端踩坑案例

- 发布事故
- 技术规范
- 稳定性手册

检索策略

采用：BM25 + Embedding 混合召回

原因：

纯向量检索问题

例如：React effect stale closure

与：useEffect 状态未更新语义接近。

但：

错误码、版本号、接口名。

Embedding 召回效果反而不好。

因此增加：关键词倒排索引提升精准度。

最终：

```
Plain Text
1  向量召回 + 关键词召回 + rerank
```

混合检索。

4. 审核后台

为什么必须人工审核

大厂内部 AI 系统通常不会：

LLM直接写入知识库

因为：

- 幻觉风险

- 错误归因
- 错误建议

可能污染团队知识。

审核后台功能

支持：

- Diff 对比
- 字段编辑
- 风险标记
- 一键发布
- 驳回重生成

技术实现

前端：

- Vue3
- Pinia
- Element Plus

后端：

- py/go/java/Node.js

Diff 高亮

为了降低审核成本：

增加：原始信息 vs AI总结

差异高亮。

审核者只需重点检查：

- 根因
- 修复方案
- 风险描述

审核时间可压缩至数分钟。

5. 自动发布链路

审核通过后：

自动：

```
Markdown编译
    ↓
Wiki API发布
    ↓
知识库索引更新
```

支持：

- Confluence
- 飞书 Wiki
- 内部 CMS

稳定性设计

1. 幂等设计

Webhook 可能重复触发。

因此：

```
PR_ID + Commit_SHA
```

生成唯一任务 Key。

避免重复入库。

2. LLM 降级机制

当：

- 模型超时
- API限流
- Token超限

时：

自动降级：结构化模板 + 规则提取

保证链路不中断。

3. 消息重试

采用：指数退避重试

避免：

- API 雪崩
- 高频重放

4. 质量评分机制

增加：

```
AI结果可信度评分
```

低于阈值：

直接进入人工重审。

评分维度：

- 字段完整率
- 语义一致性
- 历史案例匹配度

遇到的核心问题

1. LLM 幻觉严重

问题：模型会“脑补”不存在根因。

例如：

日志没有提到缓存问题，
模型却生成：

```
由于Redis缓存失效导致...
```

解决方案

增加：引用来源约束

要求：

每段结论必须绑定来源片段。

类似：

```json
{
  "root_cause": {
    "content": "...",
    "source": "PR comment #12"
  }
}
```

显著降低幻觉。

2. PR 信息噪声过大

很多 PR：

- 混合需求开发
- 同时包含重构
- Bug修复不明显

导致抽取困难。

解决方案

增加：

```
Plain Text
1  变更影响分析
```

基于：

- git diff
- 文件路径
- 错误模块

做相关性过滤。

仅保留：

```
Plain Text
1  高风险变更片段
```

给 LLM

3. Token 成本问题

如果直接喂：

- 完整日志
- 全量聊天记录

成本很高。

优化方案

增加：

日志裁剪

仅保留：

- error stack

- warning
- release window

分层摘要

先小模型压缩：日志 → 摘要

再大模型生成复盘。

整体 Token 成本下降明显。

如何进一步优化

1. Agent 化演进

未来可以从：复盘生成

升级为：故障分析Agent

自动：

- 关联监控
- 定位责任模块
- 推荐修复方案
- 生成回滚建议

2. 引入 Code Graph

当前：仅基于文本。

未来可结合：AST + 调用链

构建：代码知识图谱

提升根因定位能力。

3. 多模型路由

不同任务使用不同模型：

场景	模型
日志摘要	小模型
根因分析	大模型
分类打标	Embedding模型

降低整体成本。

项目亮点总结

1. 跑通 AI 工程闭环

事件 → 抽取 → 审核 → 发布 → 检索

就是一个完整生产链路。

2. 强调可控性

不是：AI自由生成

而是：结构化 + 审核 + 可追溯

符合企业级落地。

3. 有明确稳定性设计

包括：

- 幂等
- 降级
- 重试
- 质量评分
- Token控制

具备真实工程味道。

面试里最容易被深挖的问题

其实除了需求本身，泛 ai 八股就是高频问题

1. 为什么 RAG 不直接用向量检索?

2. 如何降低 LLM 幻觉?

3. 为什么必须人工审核?

4. 如何做长上下文压缩?

5. 如何控制 Token 成本?

6. 如何保证知识库质量?

7. 为什么采用事件驱动架构?

8. 如果每天事故量暴涨如何扩容?